JavaScript – 20 Important Questions

Output-based • Concepts • Coding | Answers inline

BASICS

Q1. What does `typeof null` return?

Answer: **"object"**. A historic bug in JS kept for backward compatibility. To check null use `x === null`.

OUTPUT

Q2. Predict the output:

```
console.log(0.1 + 0.2 === 0.3);
```

Answer: **false**. Floating-point precision: 0.1 + 0.2 = 0.30000000000000004. Fix: `Math.abs(a-b) < Number.EPSILON`.

HOISTING

Q3. Predict the output of both snippets:

```
console.log(a); var a = 5;
console.log(b); let b = 5;
```

Answer: **undefined**, then **ReferenceError**. var is hoisted and initialised to undefined; let/const are hoisted but stay in the Temporal Dead Zone (TDZ) until declared.

OPERATORS

Q4. Difference between == and ===? Output of `"5" == 5` and `"5" === 5`?

Answer: == compares after type coercion, === compares value AND type. Output: **true**, **false**. Trick: always prefer === in exams/interviews.

CLOSURES

Q5. Predict the output:

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}
```

Answer: **3 3 3**. var is function-scoped, so all callbacks share one i (=3 after loop). With **let** the output is **0 1 2** (new binding per iteration).

EVENT LOOP

Q6. Predict the output order:

```
console.log(1);
setTimeout(() => console.log(2), 0);
Promise.resolve().then(() => console.log(3));
console.log(4);
```

Answer: **1 4 3 2**. Sync code first → microtasks (Promises) → macrotasks (setTimeout). Memory trick: **S-M-M** = Sync, Micro, Macro.

COERCION

Q7. Output of `[] + []` and `[] + {}`?

Answer: **""** (empty string) and **"[object Object]"**. + converts both to strings: [] → "", {} → "[object Object]".

THIS

Q8. How does `this` behave in an arrow function vs a regular function?

```
const obj = {
  name: 'JS',
  reg() { return this.name; },
  arr: () => this.name
};
console.log(obj.reg(), obj.arr());
```

Answer: **'JS' undefined**. Regular functions get this from the caller (obj). Arrow functions have NO own this; they take it lexically from the outer scope (here module/global).

ARRAYS

Q9. Difference between `map()` and `forEach()`?

Answer: map() **returns a new array** (chainable); forEach() returns **undefined** and is used only for side effects. Neither can be stopped with break.

NAN

Q10. Output of `NaN === NaN`? How do you correctly check for NaN?

Answer: **false**. NaN is the only value not equal to itself. Use `Number.isNaN(x)` (safer than global isNaN, which coerces: isNaN('abc') → true).

COPYING

Q11. Predict the output:

```
const a = { x: { y: 1 } };
const b = { ...a };
b.x.y = 2;
console.log(a.x.y);
```

Answer: **2**. Spread makes a **shallow** copy; nested objects are still shared by reference. Deep copy: `structuredClone(a)`.

CLOSURES

Q12. Define a closure and write a counter using it.

```
function counter() {
  let count = 0;
  return () => ++count;
}
const c = counter();
c(); c(); console.log(c()); // 3
```

Answer: A **closure** is a function that remembers variables of its outer (lexical) scope even after the outer function has returned. Uses: data privacy, memoization, debounce.

PROMISES

Q13. Difference between `Promise.all` and `Promise.allSettled`?

Answer: **all**: resolves when all succeed, **rejects immediately** if any one fails (fail-fast). **allSettled**: waits for all, returns array of {status, value/reason}, never rejects. Also know: race (first settled), any (first fulfilled).

ASYNC

Q14. What does this log?

```
async function f() { return 1; }
console.log(f());
```

Answer: **Promise {<fulfilled>: 1}**. An async function **always returns a Promise**. To get 1: `f().then(console.log)` or `await f()`.

CODING

Q15. Implement a debounce function.

```
function debounce(fn, delay) {
  let timer;
  return function (...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn.apply(this, args), delay);
  };
}
```

Answer: Debounce runs fn only after **delay ms of no new calls** (e.g. search box). Throttle = run at most once per interval (e.g. scroll). Classic interview question.

CONST

Q16. Is this valid? What is printed?

```
const arr = [1, 2, 3];
arr.push(4);
console.log(arr);
arr = [5];
```

Answer: push works → **[1, 2, 3, 4]**; then **TypeError: Assignment to constant variable**. const blocks **reassignment**, not **mutation**. Use Object.freeze() to block mutation.

COERCION

Q17. Output of each:

```
console.log("3" * "4");
console.log("3" + 4);
console.log("3" - 1);
```

Answer: **12**, **"34"**, **2**. Trick: **+** with a string concatenates; **- * /** always convert to numbers.

FUNCTIONS

Q18. Difference between call(), apply() and bind()?

```
function hi(g, p) { return g + ' ' + this.n + p; }
const o = { n: 'Archit' };
hi.call(o, 'Hi', '!');
hi.apply(o, ['Hi', '!']);
const f = hi.bind(o, 'Hi'); f('!');
```

Answer: **call**: invokes now, args comma-separated. **apply**: invokes now, args as array. **bind**: returns a **new function** with fixed this (invoke later). Trick: **A**pply = **A**rray, **C**all = **C**omma.

BASICS

Q19. List all falsy values in JavaScript.

Answer: **8 falsy values**: false, 0, -0, 0n, "" (empty string), null, undefined, NaN. Everything else is truthy, including [], {}, "0" and "false".

ARRAYS

Q20. Predict the output:

```
console.log([10, 1, 2].sort());
console.log([1, 2, 3].reduce((a, b) => a + b, 0));
```

Answer: **[1, 10, 2]** and **6**. Default sort() compares as **strings**. Numeric sort: `arr.sort((a, b) => a - b)`. Always pass an initial value to reduce to avoid errors on empty arrays.